Jadavpur University

Department of Computer Science and Engineering

3rd Year, 1st Semester (Section A1, Group 2)

Systems Programming Laboratory Assignment

8086 MASM Simulation of Alphanumeric Caesar Cipher

Submitted By (Group 2):

Name	Roll Number
Sayanaditya Das	002410501008
Aditya Dutta	002410501010
Parijat Dey	002410501011
Rupam Maiti	002410501014
Soham Roy	002410501016
Shreyak Mitra	002410501027

Architecture: Intel 8086 Real Mode | **Assembler:** MASM / DOSBox

Academic Session: 2026–2027

Date: November 2, 2026

Abstract

This laboratory assignment report documents the design, implementation, and testing of a Caesar cipher simulation on the 16-bit Intel 8086 microprocessor using Microsoft Macro Assembler (MASM). The program implements full alphanumeric substitution: alphabetic letters undergo a +3 shift modulo 26, while decimal digits undergo a +3 shift modulo 10 (such that 8 maps to 1, and 9 maps to 2). Non-alphanumeric characters remain unmodified. The program architecture emphasizes modularity by encapsulating cipher transformations into dedicated subroutines (`encrypt_char` and `decrypt_char`) and centralizing configuration constants (`shift_key`, `alpha_mod`, `digit_mod`) so that a single modification adjusts the entire system. An interactive loop accepts multiple inputs until terminated by an empty line (pressing Enter).

1 Assignment Objectives and Problem Statement

The objective of this assignment is to design and develop a modular 8086 MASM program simulating the Caesar Cipher. The program fulfills the following requirements:

1. Accept plaintext strings repeatedly from the user in an interactive session.
2. Encrypt each string using a Caesar shift of +3 across alphabetic letters and numeric digits.
3. Decrypt the ciphertext to recover the original plaintext string.
4. Terminate execution cleanly when the user presses Enter on an empty line.
5. Maintain high code modularity and centralized configuration constants so that altering the shift key or moduli requires modifying only a single constant definition.

2 Mathematical Model

Let Σ_U, Σ_L, and Σ_D denote uppercase letters, lowercase letters, and decimal digits, respectively:

$$\Sigma_U = \{\text{'A'}, \ldots, \text{'Z'}\}, \quad \phi_U(x) = \text{ord}(x) - \text{ord}('A') \tag{1}$$
$$\Sigma_L = \{\text{'a'}, \ldots, \text{'z'}\}, \quad \phi_L(x) = \text{ord}(x) - \text{ord}('a') \tag{2}$$
$$\Sigma_D = \{\text{'0'}, \ldots, \text{'9'}\}, \quad \phi_D(x) = \text{ord}(x) - \text{ord}('0') \tag{3}$$

2.1 Transformation Mappings

For shift constant $K = 3$, the forward transformation $E_3(x)$ is defined piecewise:

$$E_3(x) = \begin{cases} \phi_U^{-1}\left((\phi_U(x) + K) \bmod 26\right), & x \in \Sigma_U \\ \phi_L^{-1}\left((\phi_L(x) + K) \bmod 26\right), & x \in \Sigma_L \\ \phi_D^{-1}\left((\phi_D(x) + K) \bmod 10\right), & x \in \Sigma_D \\ x, & \text{otherwise} \end{cases} \tag{4}$$

For decimal digits, the modulo 10 property provides the required wrap-around:

$$'7' \to (7 + 3) \bmod 10 =' 0'$$
$$'8' \to (8 + 3) \bmod 10 =' 1'$$
$$'9' \to (9 + 3) \bmod 10 =' 2'$$

The symmetric decryption transformation $D_3(c)$ inverts the shift:

$$D_3(c) = \begin{cases} \phi_U^{-1}\left((\phi_U(c) - K) \bmod 26\right), & c \in \Sigma_U \\ \phi_L^{-1}\left((\phi_L(c) - K) \bmod 26\right), & c \in \Sigma_L \\ \phi_D^{-1}\left((\phi_D(c) - K) \bmod 10\right), & c \in \Sigma_D \\ c, & \text{otherwise} \end{cases} \tag{5}$$

2.2 Efficient Modular Wrap-Around via Subtraction

On the Intel 8086 processor, unsigned integer division (`DIV`) consumes 80 to 90 clock cycles [1]. Because $K = 3$, values can exceed the modulus boundary by at most 3 positions:

- Letters: $\phi(x) + 3 \leq 28$. Values exceeding 25 are wrapped by subtracting 26.
- Digits: $\phi_D(x) + 3 \leq 12$. Values exceeding 9 are wrapped by subtracting 10.

During decryption, underflows below the lower boundary are corrected by adding 26 or 10. Since addition and subtraction execute in only 3 clock cycles, this substitution provides significant execution efficiency.

3 System Architecture and Modular Design

3.1 Centralized Configuration Constants

To ensure maintainability, all algorithmic parameters are declared once in the data segment:

```
1    shift_key       equ 3               ; Caesar shift distance
2    alpha_mod       equ 26              ; Alphabet modulo length
3    digit_mod       equ 10              ; Decimal digit modulo length
4    max_len         equ 64              ; Maximum buffer capacity
```

Listing 1: Centralized configuration constants in CAESAR.ASM

Updating `shift_key` from 3 to any valid offset automatically adjusts all encryption, decryption, and modular wrap-around routines without altering any loop instruction.

3.2 Interactive Input and Backspace Handling

Console input is managed through a custom `read_string` procedure using direct BIOS keyboard service `INT 16h`, Function `00h` paired with DOS character echo `INT 21h`, Function `02h`. This design resolves common emulator latency issues observed with DOS Function `0Ah` in EMU8086 and provides explicit control over editing:

- **Backspace Editing:** When Backspace (ASCII 8) is pressed, the procedure decrements the character pointer and emits the sequence `08h`, `20h`, `08h` to visually erase the character on screen.
- **Prompt Boundary Protection:** A boundary guard (`cmp si, 0`) prevents backspacing when the buffer is empty, ensuring the prompt is never erased.
- **Session Termination:** Pressing Enter (ASCII 13) on an empty input line terminates the loop and returns control to DOS via Function `4Ch`.

4 Low-Level Implementation

4.1 Modular Subroutines: `encrypt_char` and `decrypt_char`

The core cipher logic is encapsulated into independent subroutines that operate on a single character passed in register `AL`. This design decouples string iteration from transformation rules.

```
encrypt_char proc
    ; Check decimal digit: '0' <= AL <= '9'
    cmp al, '0'
    jb  enc_alpha
    cmp al, '9'
    jbe enc_is_digit

enc_alpha:
    ; Check uppercase letter: 'A' <= AL <= 'Z'
    cmp al, 'A'
    jb  enc_done
    cmp al, 'Z'
    jbe enc_is_upper

    ; Check lowercase letter: 'a' <= AL <= 'z'
    cmp al, 'a'
    jb  enc_done
    cmp al, 'z'
    ja  enc_done

enc_is_lower:
    add al, shift_key
    cmp al, 'z'
    jbe enc_done
    sub al, alpha_mod
    ret

enc_is_upper:
    add al, shift_key
    cmp al, 'Z'
    jbe enc_done
    sub al, alpha_mod
    ret

enc_is_digit:
    add al, shift_key
    cmp al, '9'
    jbe enc_done
    sub al, digit_mod

enc_done:
    ret
encrypt_char endp
```

Listing 2: Modular character encryption subroutine

Listing 2 displays the symmetric decryption routine, applying inverse operations with centralized constants.

```
decrypt_char proc
    cmp al, '0'
    jb  dec_alpha
    cmp al, '9'
    jbe dec_is_digit

dec_alpha:
    cmp al, 'A'
```

```
    jb  dec_done
    cmp al, 'Z'
    jbe dec_is_upper

    cmp al, 'a'
    jb  dec_done
    cmp al, 'z'
    ja  dec_done

dec_is_lower:
    sub al, shift_key
    cmp al, 'a'
    jae dec_done
    add al, alpha_mod
    ret

dec_is_upper:
    sub al, shift_key
    cmp al, 'A'
    jae dec_done
    add al, alpha_mod
    ret

dec_is_digit:
    sub al, shift_key
    cmp al, '0'
    jae dec_done
    add al, digit_mod

dec_done:
    ret
decrypt_char endp
```

Listing 3: Modular character decryption subroutine

4.2 Streamlined Buffer Processing Loops

With character transformations abstracted into modular subroutines, string processing in `main` becomes clean and concise:

```
    ; Encryption loop
    xor cx, cx
    mov cl, actual_len
    xor si, si
encrypt_loop:
    mov al, plaintext[si]
    call encrypt_char
    mov encrypted_str[si], al
    inc si
    loop encrypt_loop

    ; Decryption loop
    xor cx, cx
    mov cl, actual_len
    xor si, si
decrypt_loop:
    mov al, encrypted_str[si]
    call decrypt_char
    mov decrypted_str[si], al
    inc si
    loop decrypt_loop
```

Listing 4: Main loop string processing

5 Experimental Evaluation

The implementation was executed and tested within the DOSBox 0.74-3 emulator environment.

5.1 Interactive Multi-Turn Console Session

The listing below shows an interactive test session verifying letter shifts, digit wraps, mixed strings, and exit handling:

```
=== 8086 Caesar Cipher Simulator (+3 Shift) ===
Enter string to transform, or press ENTER on empty line to exit.

Enter plaintext string: HELLO
--- Cipher Results ---
Original Plaintext  : HELLO
Encrypted (+3 Shift): KHOOR
Decrypted Plaintext : HELLO

Enter plaintext string: CS101 Room 89!
--- Cipher Results ---
Original Plaintext  : CS101 Room 89!
Encrypted (+3 Shift): FV434 Urrp 12!
Decrypted Plaintext : CS101 Room 89!

Enter plaintext string: 789 XYZ xyz
--- Cipher Results ---
Original Plaintext  : 789 XYZ xyz
Encrypted (+3 Shift): 012 ABC abc
Decrypted Plaintext : 789 XYZ xyz

Enter plaintext string:
Empty input detected (ENTER pressed). Program terminated.
```

5.2 Test Suite and Validation Matrix

Table 1 documents the empirical test vectors verifying all boundary conditions.

Table 1: Empirical Test Results Across Character Classes and Boundary Conditions

Test Category	**Plaintext (p)**	**Ciphertext ($E_3(p)$)**	**Verified Behavior**
Standard Text	`HELLO`	`KHOOR`	Uppercase shift without wrap
Alphabet Wrap	`XYZxyz`	`ABCabc`	Upper and lowercase boundary wrap
Digit Wrap	`789`	`012`	Modulo 10 wrap ($8 \to 1, 9 \to 2$)
Alphanumeric Mixed	`CS101 Room 89`	`FV434 Urrp 12`	Simultaneous letter and digit shifts
Punctuation and Spaces	`A+B = 9;`	`D+E = 2;`	Operators and punctuation preserved
Exit Condition	`(empty)`	`(none)`	Program terminates cleanly on bare Enter

6 Performance and Complexity Analysis

6.1 Complexity Characteristics

- **Time Complexity:** For an input of length $N \leq 64$, both encryption and decryption require $\Theta(N)$ operations. Each character is processed in bounded $O(1)$ time within `encrypt_char` and `decrypt_char`.
- **Space Complexity:** Data segment usage is under 250 bytes with zero dynamic heap allocation, remaining well within the 64 KB segment.

6.2 Instruction Cycle Timing Comparison

Table 2 compares instruction cycle requirements between subtraction-based modulo reduction and hardware division.

Table 2: Instruction Cycle Requirements for 8086 Modular Reduction

Method	**Instructions Executed**	**Clock Cycles**
Subtraction-Based (Implemented)	`ADD AL, shift_key`	3
	`CMP AL, '9'`	3
	`JBE enc_done` (no wrap)	4
	`SUB AL, digit_mod` (wrap taken)	3
	Worst-Case Total	**13 cycles**
Division-Based (Alternative)	`SUB AL, '0'`	3
	`ADD AL, 3`	3
	`MOV CL, 10` / `CBW`	4
	`DIV CL`	80–90
	`ADD AH, '0'`	3
	Average Total	**93–103 cycles**

The subtraction approach yields an approximate 7× speedup over the division instruction, demonstrating effective assembly-level optimization.

7 Conclusion

This assignment report documented the simulation of an alphanumeric Caesar cipher on the Intel 8086 microprocessor. Character transformations are abstracted into modular subroutines `encrypt_char` and `decrypt_char`. Centralized configuration constants allow adjusting the shift distance from a single definition. The program processes uppercase letters, lowercase letters, and digits (wrapping 8 to 1, and 9 to 2) while preserving symbols. An interactive console loop supports continuous testing and terminates cleanly upon pressing Enter.

References

[1] Intel Corporation, *The 8086 Family User's Manual*, Santa Clara, CA: Intel Corporation, Oct. 1979.

[2] D. V. Hall, *Microprocessors and Interfacing: Programming and Hardware*, 3rd ed., New Delhi, India: Tata McGraw-Hill, 2005.

[3] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 7th ed., Boston, MA: Pearson, 2017.

[4] P. Abel, *IBM PC Assembly Language and Programming*, 5th ed., Upper Saddle River, NJ: Prentice Hall, 2001.

[5] A. K. Ray and K. M. Bhurchandi, *Advanced Microprocessors and Peripherals: Architecture, Programming and Interfacing*, 3rd ed., New Delhi, India: McGraw-Hill Education, 2012.